AI Engineering and Claude Architect · Part IV, Advanced

Advanced AI Topics, Lesson 1

Fine-Tuning, LoRA and When Not to Bother

A team spent six weeks fine-tuning a model to learn their product catalogue. Then the catalogue changed. Fine-tuning teaches behaviour and style, and it's a poor place to keep facts.

Module	AI11 of 13
Lesson	1 of 5
Prerequisites	AI01 L3, AI02 L4, AI03 L3, AI05 L1, AI10 L3
Exam relevance	Out of scope. The exam prefers prompts and few-shot examples (TS 4.2)
Reading time	~30 minutes

Contents

1. Fine-Tuning and LoRA

Lesson 1 of 5 · Advanced AI Topics

Fine-Tuning and LoRA

Key idea

Fine-tune last, and only for repeatable behaviour that prompts, few-shot examples and retrieval have failed to deliver, and that evals can measure. Facts belong in retrieval, as they change. LoRA makes fine-tuning cheap by training a small update beside frozen weights, and a small update can also generalize better from little data.

> *Before: gradient descent, how LLMs are trained, prompting, RAG and evals. Teaches: what fine-tuning changes, the ladder of cheaper alternatives, how LoRA and QLoRA work, data needs, poisoning, and costs that come later. Used by: L2.*

The hook: teaching a model facts

An invented story. A retailer fine-tunes a model on 40,000 product descriptions, to help it "know the catalogue." Six weeks and several training runs later, it answers well in the demo. Then:

Week	What happened
1	Prices change. The model gives old prices, confidently
3	New products launch. It invents details about them
8	The base model is retired, so the fine-tune has to be redone
12	Nobody can say why an answer is wrong: there's no source to check

RAG would have answered all of this on day one, with a citation. The team confused two kinds of knowledge:

Kind	Example	Where it belongs
Facts that change	Prices, policies, stock	Retrieval or tools (AI05)
Rules you can state	"Use this format," "escalate if…"	System prompt (AI03)
Judgment shown by examples	Which tickets are urgent	Few-shot examples (TS 4.2)
Repeated style or a narrow skill	House tone, one classification, at scale	**Possibly fine-tuning**

Intuition: hiring and training

You have a capable new employee. To get better work from them, you could write them a clear brief, or show them some examples, or give them access to the filing system. All of that works from day one, and you can change it tomorrow.

Or you can send them on a six-week course, which changes how they work. It's expensive, and when the job changes you have to send them again. It's worth doing for a skill they'll use every day, and a waste for facts they could look up.

Mechanism

What fine-tuning changes

AI02 L4 taught training. Pre-training teaches a model language, while fine-tuning continues training on your examples, nudging the weights towards your task.

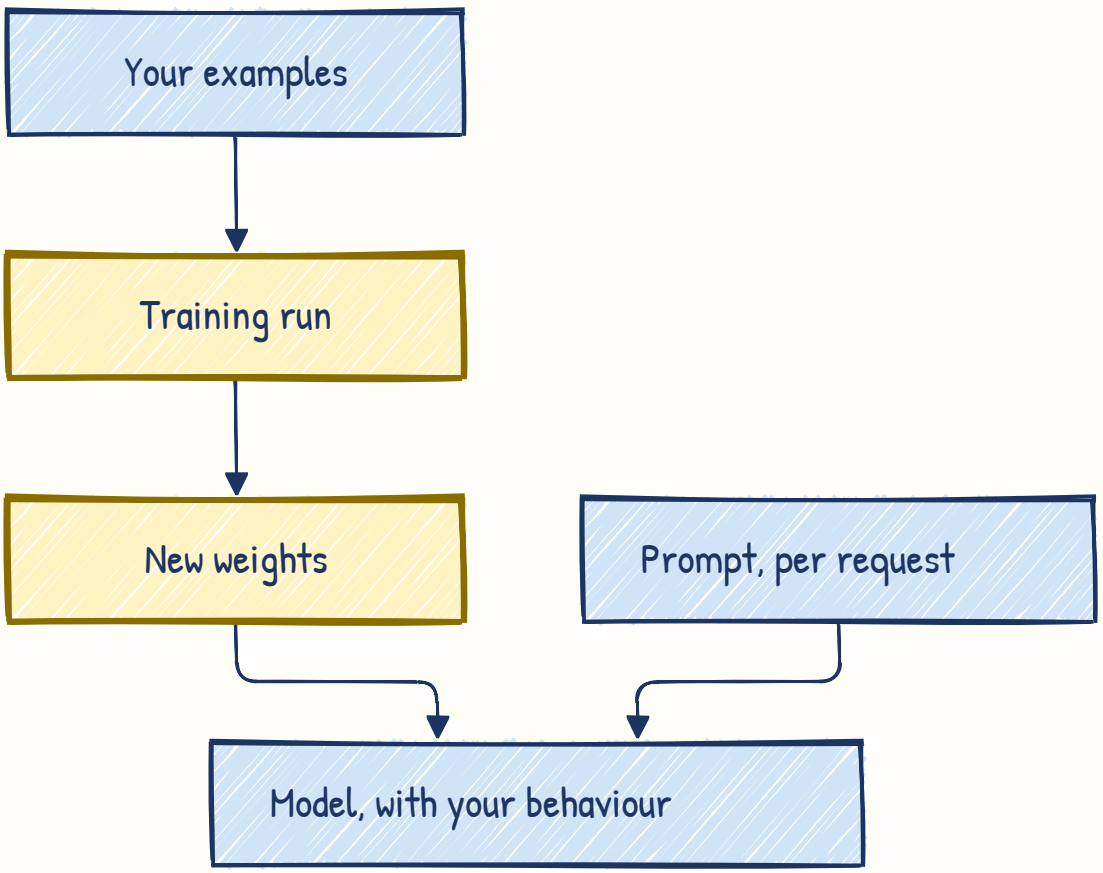


Figure 1. Prompts change the input on every call. Fine-tuning changes the weights once, and every call afterwards depends on them.

Weights are fixed after training (AI02 L3). So a fine-tuned model doesn't update itself as your data changes. That's the root of the hook's trouble.

The ladder: try these first

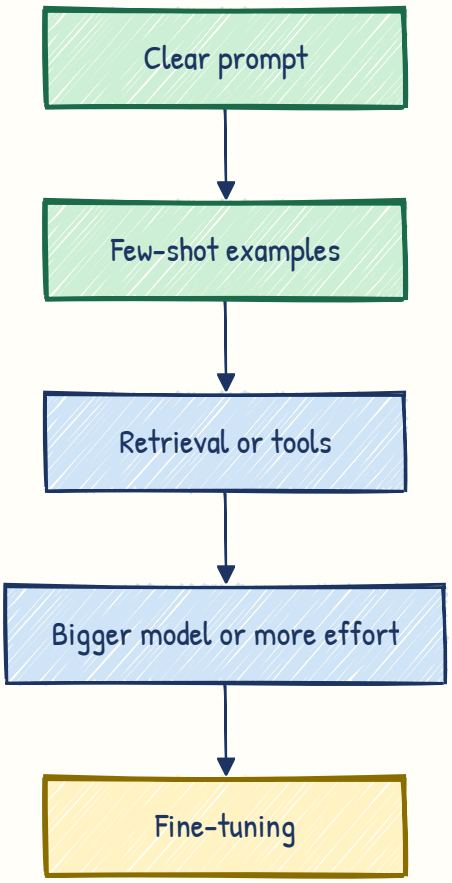


Figure 2. Each rung costs more, and takes longer to change. Measure with evals at each one, and stop when the target is met.

Rung	Time to change	Cost to change	Note
Prompt	Minutes	Nearly nothing	Start here
Few-shot examples	Minutes	Input tokens, and caching helps	TS 4.2
Retrieval or tools	Days	Index upkeep	For facts
Bigger model, more effort	Minutes	Per-token	AI03 L4
Fine-tuning	Days to weeks	Data, training, evals, re-training	Last

When fine-tuning can be right

Good reason	Why
Very high volume, narrow task	A long prompt repeated billions of times can cost more than training. Check caching first
Smaller model doing a bigger model's job	Cost and latency, if evals agree
Consistent format or style	Examples may be better taught than described
A language or domain the base handles poorly	Prompts can't add what the model lacks. Evidence required
Offline or self-hosted	You own the weights (L2)

Anthropic's published example. Anthropic announced fine-tuning for Claude 3 Haiku through Amazon Bedrock in July 2024, generally available that November. The post names the same kinds of use: classification, custom API interactions, and industry-specific data interpretation. It reports an internal moderation test in which accuracy rose from 81.5% to 99.6%, with tokens per query down 85%. **These are vendor figures on one task, from 2024.** The Claude API docs I've read for this course offer no fine-tuning endpoint for current models, so check what's available before planning on it.

How LoRA works

Full fine-tuning updates every weight. A large model has billions, so this is costly, and storing a full copy per task is worse.

LoRA (Hu et al.) observes that the change a task needs is often simple. Freeze the original matrix `W`, and learn two small matrices whose product is the change:

`W' = W + A × B`, where `A` is `d × r`, `B` is `r × d`, and the rank `r` is small.

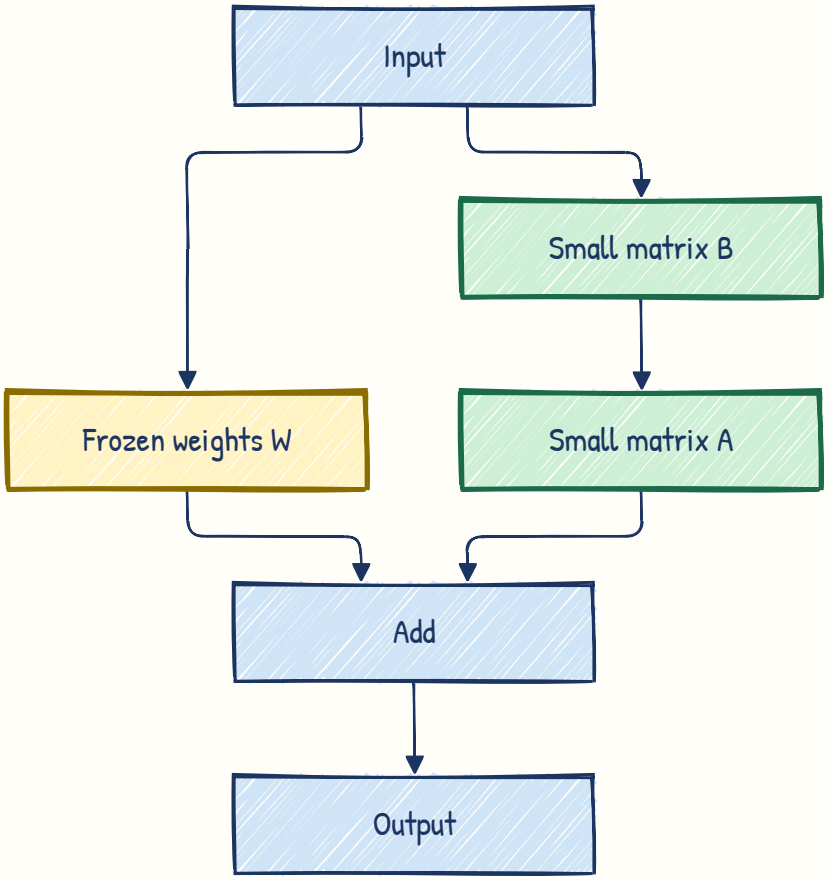


Figure 3. The base weights never change. Only the small adapter trains, and several adapters can share one base.

The saving is large, and easy to compute. Real output from `LoraLab.java`:

```
LoRA trainable parameters for one 4096 x 4096 matrix:
  rank  8:    65,536 of 16,777,216, or 0.39%
  rank 16:   131,072 of 16,777,216, or 0.78%
  rank 64:   524,288 of 16,777,216, or 3.13%
```

The paper's abstract reports, against fine-tuning GPT-3 175B with Adam, 10,000 times fewer trainable parameters, 3 times less GPU memory, and results that match or beat fine-tuning on the models tested, with no added inference latency.

QLoRA (Dettmers et al.) goes further: it freezes the base at 4 bits, and trains adapters on top. The abstract describes fine-tuning a 65B model on one 48 GB GPU. Those are its authors' claims on their benchmark, and it adds that chatbot benchmarks are unreliable. L2 covers quantization.

A small adapter can generalize better

I trained this for real, in plain Java, on a 40 × 40 layer. The new task differs from the old by a change of rank 2. **This is a toy linear model, chosen to suit low rank.** Real output:

```
30 training examples (test error, lower is better):
  no training           params     0  error 1.4913
  full fine-tune        params  1600  error 0.4327
  LoRA rank 1           params    80  error 0.8774
  LoRA rank 2           params   160  error 0.2934
  LoRA rank 8           params   640  error 0.3126

100 training examples (test error, lower is better):
  no training           params     0  error 2.1575
  full fine-tune        params  1600  error 0.0569
  LoRA rank 1           params    80  error 0.8275
  LoRA rank 2           params   160  error 0.0061
  LoRA rank 8           params   640  error 0.0089

400 training examples (test error, lower is better):
  no training           params     0  error 2.0721
  full fine-tune        params  1600  error 0.0103
  LoRA rank 1           params    80  error 0.7534
  LoRA rank 2           params   160  error 0.0023
  LoRA rank 8           params   640  error 0.0058
```

Three readings:

→ **With 100 examples, rank 2 beats full fine-tuning by about 9 times,** using 10% of the parameters. Fewer parameters have less room to fit noise (AI01 L2).

→ **Rank 1 is too small,** and more data doesn't rescue it. Too small a rank can't express the change.

→ **Too large is a milder mistake.** Rank 8 costs a little accuracy here.

So rank is a hyperparameter to tune on held-out data. **Choose the smallest adapter that passes your evals.**

Data is the real work

Requirement	Why
Examples of the actual task	Mirrors production (AI10 L3)
Labels you trust	The model learns your mistakes as well
A held-out test set	Otherwise you measure memorization
Baseline with no fine-tune	Prompted base model is the bar to beat
Segment coverage	Rare cases need examples
Permission to use the data	Personal data in weights can't practically be deleted (AI09 L4)

Poisoning: a few documents can be enough

If outsiders can influence training data, consider this. Anthropic, the UK AI Security Institute, and the Alan Turing Institute published a study in October 2025. In it, as few as 250 malicious documents implanted a backdoor, for models from 600M to 13B parameters, with success tracking the absolute count, and not the share of data. Its limits are clear: the backdoor made the model output gibberish on a trigger, a narrow and low-stakes behaviour, and it's unclear whether this holds for larger models or more harmful behaviours.

A toy version, with a logistic classifier in which each example is seen the same number of times. **It's a linear toy, and shows only why count can matter more than share:** a rare trigger's weight is learned from the few examples containing it.

```
Toy backdoor: share of class-0 inputs flipped when the trigger appears
    1,000 clean:    0 poison   1.3%    5 poison  64.4%   25 poison  89.2%  100 poison  98.2%
   10,000 clean:    0 poison   0.8%    5 poison  41.3%   25 poison  78.1%  100 poison  94.4%
  100,000 clean:    0 poison   0.4%    5 poison  21.4%   25 poison  65.8%  100 poison  89.1%
```

With 100 poisoned examples, the clean data grew by 100 times and the attack fell only from 98% to 89%. Dilution did little. At lower counts, more clean data does help more in this toy. **Don't rely on your dataset's size to protect it.** Track where training data came from, review it, and test for odd triggers.

Later costs

Later cost	Why
Re-training on model retirement	A fine-tune is tied to its base (AI10 L6)
Missing later improvements	A newer base may beat your tuned old one
Eval upkeep	Each new version needs measuring
Safety regression	Training changes behaviour, and refusals can weaken. Test it. This is general knowledge, and I read no specific study for this course
Debugging	Weights give no citation

Why the obvious fixes fall short

Obvious idea	Why it falls short
Fine-tune to teach facts	Stale immediately, and no source
Fine-tune because the prompt is long	Caching makes long prompts cheap (AI10 L5)
More data always fixes it	Rank 1 stayed poor at 400 examples
Full fine-tune for best quality	Rank 2 did better in the toy, and on the paper's benchmarks LoRA matched it
A huge dataset is safe from poisoning	Count matters, more than share
Use the demo as evidence	Needs a held-out eval, against the prompted baseline

How Anthropic and the field approach it

Anthropic. The docs emphasize prompting, few-shot examples and evaluation first (AI03). Its fine-tuning offer is the 2024 Haiku post above, and I haven't confirmed its current state. Don't present fine-tuning as a recommended Claude path without checking.

The field. LoRA and QLoRA are the common methods for open models. Python tools dominate training (AI02 L4), and I ran none of them for this course. My lab is a small linear model in Java, for understanding only.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a prompted API model.

Improved: one fine-tuned adapter on an open base. New problem: nobody can reproduce it, and when the base changes it's stranded.

Production: a pipeline, in which an adapter is a versioned artifact with a data lineage and an eval.

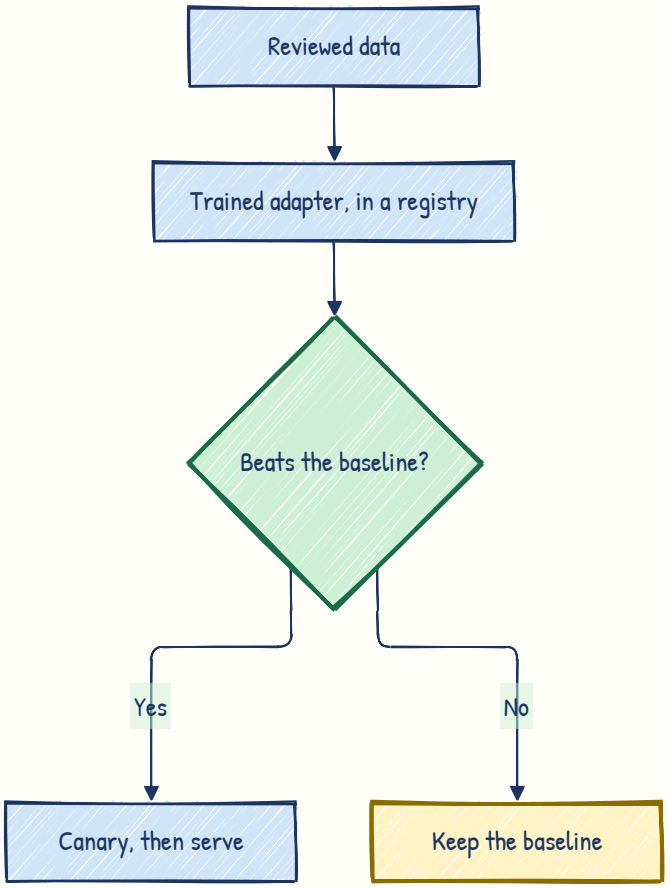


Figure 4. An adapter is released like code. It only ships if it beats the prompted baseline on held-out data.

Component	Why it exists	How it communicates
Data store	Lineage and review, including poisoning checks	Versioned files
Training job	Reproducible, on rented GPUs	Scheduler
Model registry	Adapter, base version, data version, metrics	API
Eval gate	Compares with the prompted baseline, by segment	CI (AI10 L3)
Serving	Loads the base plus adapters	L2
Monitor	Drift, and whether re-training is due	AI10 L4

Failures:

Failure	How you'd notice	What to do
Overfit to training data	Train high, test low	Smaller rank, more data, earlier stop
Stale behaviour	Drift, and eval decay	Re-train, or move the content to retrieval
Poisoned data	Odd outputs on a trigger	Lineage, review and targeted tests
Base retired	Calendar alert	Re-train and re-evaluate
Worse refusals	Safety evals	Add safety cases to the gate
Private data memorized	Extraction tests	Clean data before training

Java

Training lives in Python, usually. Understanding it doesn't need that. This is the whole LoRA forward pass in my lab, with the frozen base added to the low-rank part:

```
out[i] += a[i][r] * bx[r];
```

And the update touches only `A` and `B`. The base has no update line at all, and that's the point:

```
a[i][r] -= rate * diff[i] * bx[r];
```

In a Java service, an adapter is a deployment detail. You call an inference server over HTTP (AI02 L3), and it loads the adapter.

Trade-offs

Decision	Benefit	Cost
Fine-tune	Baked-in behaviour, perhaps shorter prompts	Data, training and re-training
Prompt and few-shot	Instant change	Input tokens, on each call
LoRA	Cheap and small. Many tasks share a base	Limited by rank, and needs an open base
Full fine-tune	Most flexible	Cost, storage, overfitting
Retrieval	Fresh facts, with sources	Index, and retrieval errors

When not to: nearly always, at first. If you can't show an eval where the prompted baseline fails and the fine-tune wins, don't do it.

Common mistakes

Fine-tuning before you have an eval. You can't show improvement, or notice regression.

Common mistakes

Comparing a fine-tune to a poor prompt. Compare with the best prompted baseline you can write, including few-shot examples.

What to say in an interview

"I'd climb a ladder: prompt, few-shot, retrieval for facts, better model or effort, then fine-tuning. It's for repeatable behaviour at scale, proven by evals against the prompted baseline. Facts go in retrieval because they change. If I do tune, I'd use LoRA on an open base and treat the adapter as a versioned artifact, with data lineage, since a small number of poisoned documents can matter, and it has to be redone when the base retires."

Certification lens

How this lesson maps to CCAR-F

Out of scope. The exam guide doesn't test fine-tuning or LoRA. Its nearby idea is TS 4.2: using few-shot examples for ambiguous cases. In an exam question, expect prompt-level fixes to be the right answer.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Structured Data Extraction · Medium. An extractor handles most documents, yet it's inconsistent on ambiguous cases, such as which date is the "effective date." What should you try first?

A. Fine-tune a model on 10,000 labelled documents

B. Raise `max_tokens`

C. Remove the ambiguous fields from the schema

D. Add a few targeted examples showing the reasoning on those ambiguous cases

A D (TS 4.2). Few targeted examples are quick to try and to measure. A is costly and premature. B has nothing to do with it, and C loses required data.

Q Q2 · A colleague proposes fine-tuning to teach a model your refund policy, which changes quarterly. Respond.

A Policy is a fact that changes, so it belongs in retrieval or the system prompt, where it's updated instantly and can be cited. A fine-tune would go stale, couldn't show its source, and would need re-training each quarter. Reconsider only if evals show a repeated behaviour that prompts can't achieve.

Q Q3 · Why can a low-rank adapter beat full fine-tuning on a small dataset?

A It has far fewer parameters, so it has less room to fit noise. If the required change really is simple, the small adapter can express it and generalizes better. In my toy with 100 examples, rank 2 had about nine times lower error than full fine-tuning. If the rank is too small, it can't express the change at all.

Revision - Lesson 1

→ Facts go in retrieval, rules in the prompt, judgment in few-shot examples. Style and narrow skills may suit fine-tuning.

→ Weights are fixed after training, so a fine-tune goes stale.

→ Climb the ladder: prompt, few-shot, retrieval, model or effort, then fine-tuning.

→ LoRA freezes the base, and trains a small `A × B` update.

→ A 4096-square matrix at rank 8 trains 0.39% of the parameters.

→ QLoRA trains adapters on a 4-bit base, per its abstract.

→ Rank is a hyperparameter. Too small can't express the change, and too large overfits a bit.

→ Anthropic's study: about 250 documents backdoored models regardless of size, for a narrow behaviour.

→ Costs arrive later: re-training on retirement, evals and debugging.

→ Only ship an adapter that beats the prompted baseline on held-out data.

Your notes